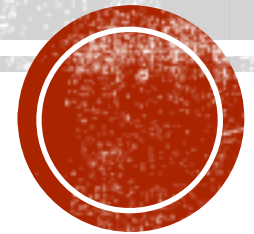


POINTEURS ET LISTES CHÂÎNÉES



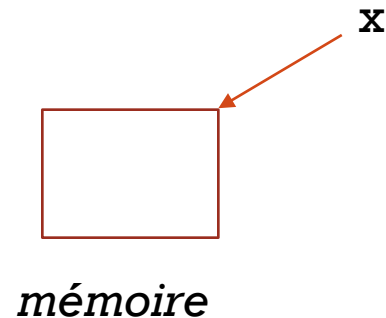
NOTION DE VARIABLE

Exemple:

Variable x: entier;

s: caractère;

- Toute variable manipulée dans un programme est stockée quelque part en mémoire centrale.



- Deux types de variables: **statiques** et **dynamiques**



VARIABLES STATIQUES

- La place réservée en mémoire à une variable statique ne change pas au cours de l'exécution du programme.
- Risques
 - i. Si l'espace mémoire réservée est **plus grande** que celle qui est effectivement utilisée par le programme, cela peut entraîner un gaspillage de place
 - ii. Si l'espace mémoire réservé est **trop petite**, il faut alors que le programme prenne en charge le contrôle du débordement: manque place
- Pour retrouver une variable, il suffit de connaître son adresse.
- Pour des raisons de **lisibilité**, on désigne souvent les variables par des identificateurs et non par leurs adresses.
- C'est le **compilateur** qui fait alors le **lien** entre **l'identificateur d'une variable** et son **adresse** en mémoire.
- Parfois, il est très pratique de manipuler une variable par son adresse: c'est l'utilité des **pointeurs**.

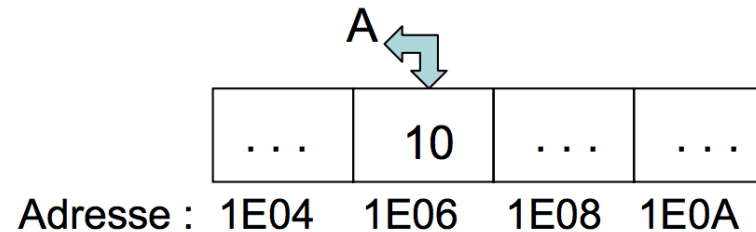


ADRESSAGE DIRECT

- Jusqu'à maintenant, nous avons surtout utilisé des variables pour stocker des informations.
- La valeur d'une variable se trouve à un endroit spécifique dans la mémoire de l'ordinateur.

A: entier

$A \leftarrow 10$

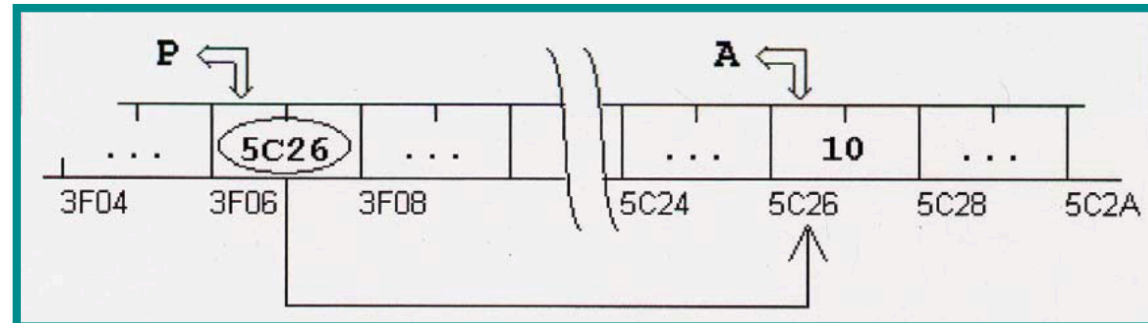


- Le nom de la variable nous permet alors d'accéder directement à cette valeur.
- **Définition:** dans l'**adressage direct**, l'accès au contenu d'une variable se fait via le nom de la variable.



ADRESSAGE INDIRECT

- Si nous ne voulons ou ne pouvons pas utiliser le nom d'une variable A, nous pouvons copier l'adresse de cette variable dans une variable spéciale, disons P, appelée pointeur.
- Nous pouvons alors retrouver l'information de la variable A en passant par le pointeur P.
- **Définition:** dans l'**adressage indirect**, l'accès au contenu d'une variable se fait via un pointeur qui renferme l'adresse de la variable.



VARIABLES DYNAMIQUES

- Parfois, il est très pratique de manipuler une variable par son adresse: c'est l'utilité des **pointeurs**.
- Les variables **dynamiques**, appelées **pointeurs** vont permettre:
 - i. d'allouer de l'espace mémoire au fur et à mesure des besoins dans la limite de l'espace disponible
 - ii. de libérer de l'espace non utilisé.
- **Définition**: un pointeur est une variable dont la valeur est égale à l'adresse d'un autre objet.
- Importance des pointeurs:
 - Programmes plus efficaces
 - Utilisation de nouvelles structures de données: **listes chaînées**, **piles**, **files**



DÉCLARATION D'UN POINTEUR

`nom_du_pointeur: *Type`

`nom_du_pointeur`: identificateur dont la valeur est l'adresse d'un objet de type `Type`

`Type`: type de la variable pointée

Remarques

- L'opérateur unaire d'indirection `*` permet d'accéder directement à la valeur de l'objet pointé.
- Ainsi si `p` est un pointeur vers un entier `i`, `*p` désigne la valeur de `i`.
- Pour obtenir l'adresse d'une variable, on utilise l'opérateur `&` précédant le nom de la variable.

`&nomVariable`



EXEMPLE

Variables $p:*\text{entier};$
 $i: \text{entier};$

$p \leftarrow \&i;$

ça correspond à:

$*p$ désigne la valeur de i

p désigne l'adresse de i

Variables $A, B : \text{entier};$
 $p: *\text{entier};$

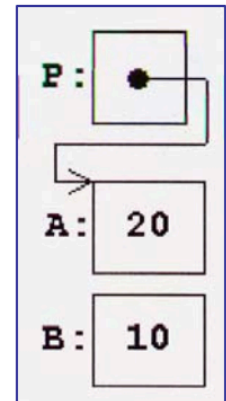
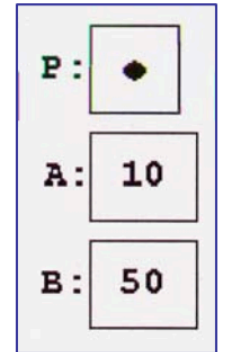
$A \leftarrow 10$

$B \leftarrow 50$

$P \leftarrow \&A;$

$B \leftarrow *P;$

$*P \leftarrow 20$



NB: $*P$ et A désigne le même emplacement mémoire et $*P$ peut être utilisé partout où on peut écrire A



EXEMPLE

Variables X, Y : entier;

P:*entier;

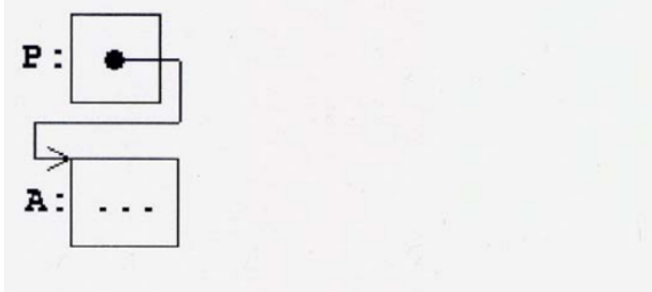
P←&X;

Y← *P+1 est équivalent à Y←X+1

*P←*P+10 est équivalent à X←X+10



EN RÉSUMÉ



A désigne le contenu de A

&A désigne l'adresse de A

P désigne l'adresse de A

***P** désigne le contenu de A

En outre:

&P désigne l'adresse du pointeur P

***A** est illégal (puisque A n'est pas un pointeur)

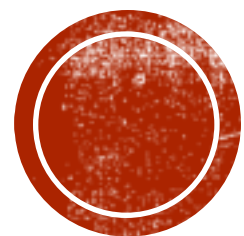


LE POINTEUR NULL

- Pour indiquer qu'un pointeur pointe nulle part, on utilise l'identificateur NIL

$P \leftarrow \text{NIL};$





POINTEURS ET TABLEAUX



POINTEURS ET TABLEAUX

- Chaque opération avec des indices de tableaux peut aussi être exprimée à l'aide de pointeurs.
- Le nom d'un tableau est un pointeur **constant** sur le premier élément du tableau.
- Le nom d'un tableau représente l'adresse de la première composante.

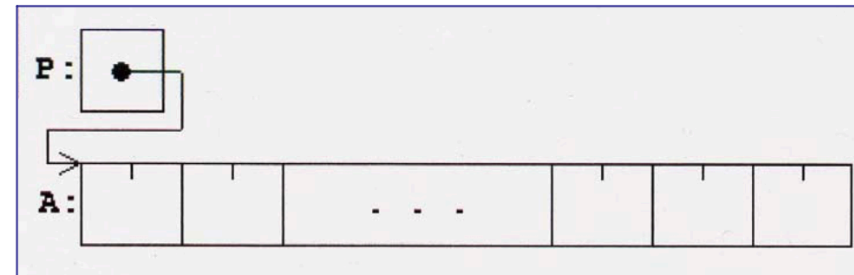
$\&A[1]$ et A sont une seule et même adresse.

Exemple

Variables:

A : tableau[1..10] d'entier;

p :*entier;



$P \leftarrow A$; est équivalent à $P \leftarrow \&A[1]$;



ADRESSAGE DES COMPOSANTES D'UN TABLEAU

Si P pointe sur une composante quelconque d'un tableau, alors $P + 1$ pointe sur la composante suivante.

Ainsi, après l'instruction $P = A;$

$*(P)$ désigne le contenu de $A[1]$

$*(P+1)$ désigne le contenu de $A[2]$

...



EN RÉSUMÉ

A	désigne l'adresse de	$A[1]$
$A+i$	désigne l'adresse de	$A[i+1]$
$*(A+i)$	désigne le contenu de	$A[i+1]$

Si $P \leftarrow A$, alors

P	pointe sur l'élément	$A[1]$
$P+i$	pointe sur l'élément	$A[i+1]$
$*(P+i)$	désigne le contenu de	$A[i+1]$



TABLEAU DE POINTEURS

Variable

tab : Tableau[1..N] de *Type;

C'est la déclaration d'un tableau **NomTableau** de **N** pointeurs sur des données du type **Type**.

Exemple

Variable T:tableau [1..10] de *entier;

T est un tableau de 10 pointeurs vers des valeurs de type **entier**

NB: Se référer au chapitre sur les tableaux pour les manipulations possibles



ALLOCATION DYNAMIQUE

- Lorsque l'on déclare une variable caractère, entier, réel, ... un nombre de cases mémoire bien défini est réservé pour cette variable. Ce n'est pas le cas avec les pointeurs.
- Avant de manipuler un pointeur, il faut l'**initialiser** soit par une allocation dynamique soit par une affectation d'adresse $p \leftarrow \&i$.
- Sinon, par défaut la valeur du pointeur est égale à une constante symbolique notée **NIL**.
- L'allocation se fait avec la fonction **Allouer**



EXEMPLE

Algorithme Exemple1

Variables

pi,pj : *Entier

pr : *Réal

pc : *Caractère

Début

//réserver 10 octets mémoire, soit la place pour 10 caractères

pc←Allouer(10)

//réserver un espace pour 4 entiers

pi←Allouer(4)

//réserver un espace pour 6 réels

pr←Allouer(6)

//réserver un espace pour 1 entiers

pj←Allouer() // Par défaut le nombre d'objet est 1

Fin



LIBÉRER LA MÉMOIRE

Algorithme Remarque 1

Variable

i : Entier

p : *Entier

Debut

p ← &i ;

Fin

i et *p sont identiques et on n'a pas besoin d'allocation dynamique puisque l'espace mémoire à l'adresse &i est déjà réservé pour un entier.

Lorsque l'on n'a plus besoin de l'espace mémoire allouée dynamiquement c'est-à-dire quand on n'utilise plus le pointeur p, il faut libérer cette place en mémoire. Ceci se fait à l'aide de l'instruction **Libérer** qui a pour syntaxe :

Libérer(nom_du_pointeur)



EXEMPLE

Algorithme Exemple1

Variable

pi : *Entier

pr : *Réal

Début

//réserver un espace pour 4 entiers

pi ← Allouer(4)

//réserver un espace pour 6 réels

pr ← Allouer(6)

//libérer la place précédemment réservée pour pi

Libérer(pi)

//libérer la place précédemment réservée pour pr

Libérer(pr)

Fin



LES POINTEURS EN RÉSUMÉ

Bonne pratique de programmation: optimiser les ressources (mémoires, CPU)

- maîtriser les déclarations de variables, comment ?
- Utilisant des variables dynamiques: allouer et libérer



Les pointeurs

- Usage des pointeurs: les structures de données [Listes chaînées](#), [Piles](#), [Files](#)



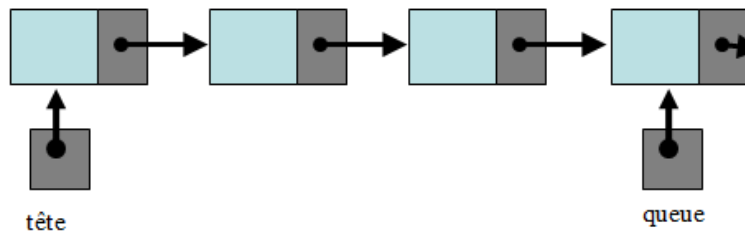
LISTES CHAINÉES



DÉFINITION

- Une liste chaînée est une structure qui permet de stocker de manière ordonnée des éléments.
- Une LC est composée de maillons.
- Un maillon est une structure qui contient un élément à stocker et un pointeur sur le prochain maillon de la liste.

Définition: une liste chaînée est une succession de maillons, dont le dernier pointe vers une adresse invalide (NIL).



STRUCTURE MAILLON

```
{  
    Element : type  
    Suivant : ^MAILLON /*Pointeur vers le maillon suivant */  
}
```



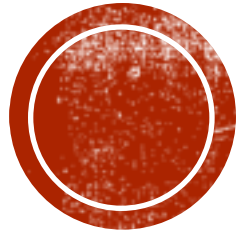
DÉCLARATION D'UNE LC

- Une liste chaînée est caractérisée par un pointeur **tete** (ou *premier*) vers le premier élément et un pointeur **queue** (ou *dernier*) vers le dernier élément de la liste.

STRUCTURE LISTE

```
{  
    Tete   : ^MAILLON  
    Queue: ^MAILLON  
}
```





MANIPULATIONS DES LISTES CHAÎNÉES

Créer une liste. Ajouter un élément. Supprimer un élément. Modifier un élément. Parcourir une liste. Rechercher une valeur dans une liste.

CRÉER UNE LISTE CHAINÉE

```
Type Liste = ^Element;  
  
Type Element = Structure  
    Info : chaîne de caractères  
    Suivant : Liste  
  
Fin
```

Algorithme CréationListe2Elements

Tete, P : Liste
NombreElt : entier

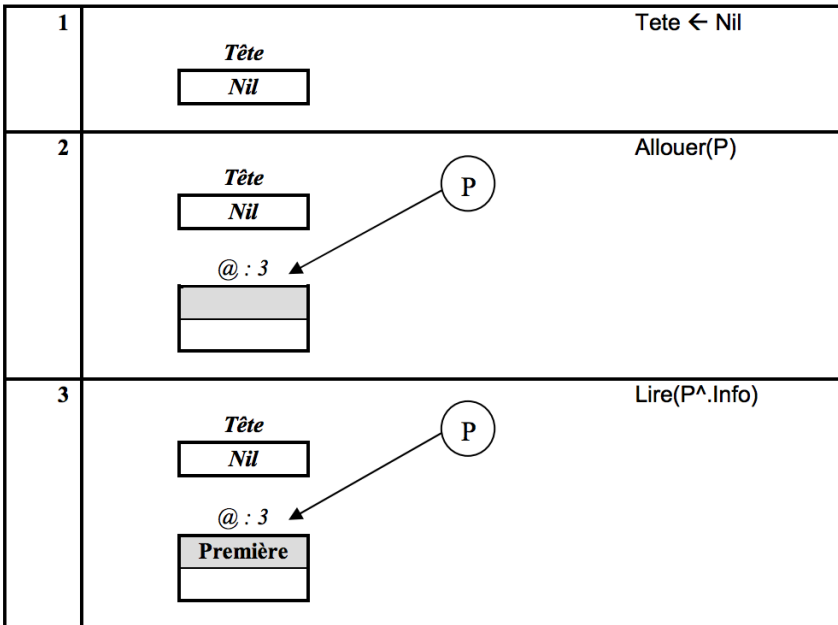
DEBUT

```
Tete ← Nil          /* pour l'instant la liste est vide */  
Allouer(P)          /* réserve un espace mémoire pour le premier élément */  
Lire(P^.Info)       /* stocke dans l'Info de l'élément pointé par P la valeur saisie */  
P^.Suivant ← Nil    /* il n'y a pas d'élément suivant */  
Tete ← P            /* le pointeur Tete pointe maintenant sur P */  
/* Il faut maintenant ajouter le 2e élément, ce qui revient à insérer un élément en  
tête de liste */  
Allouer(P)          /* réserve un espace mémoire pour le second élément */  
Lire(P^.Info)       /* stocke dans l'Info de l'élément pointé par P la valeur saisie */  
P^.Suivant ← Tete   /* élément inséré en tête de liste */  
Tete ← P
```

FIN



CRÉER UNE LISTE CHAÎNÉE



Algorithme CréationListe2Elements

Tete, P : Liste

NombreElt : entier

DEBUT

Tete ← Nil /* pour l'instant la liste est vide */

Allouer(P) /* réserve un espace mémoire pour le premier élément */

Lire(P^.Info) /* stocke dans l'Info de l'élément pointé par P la valeur saisie */

P^.Suivant ← Nil /* il n'y a pas d'élément suivant */

Tete ← P /* le pointeur Tete pointe maintenant sur P */

/* Il faut maintenant ajouter le 2e élément, ce qui revient à insérer un élément en tête de liste */

Allouer(P) /* réserve un espace mémoire pour le second élément */

Lire(P^.Info) /* stocke dans l'Info de l'élément pointé par P la valeur saisie */

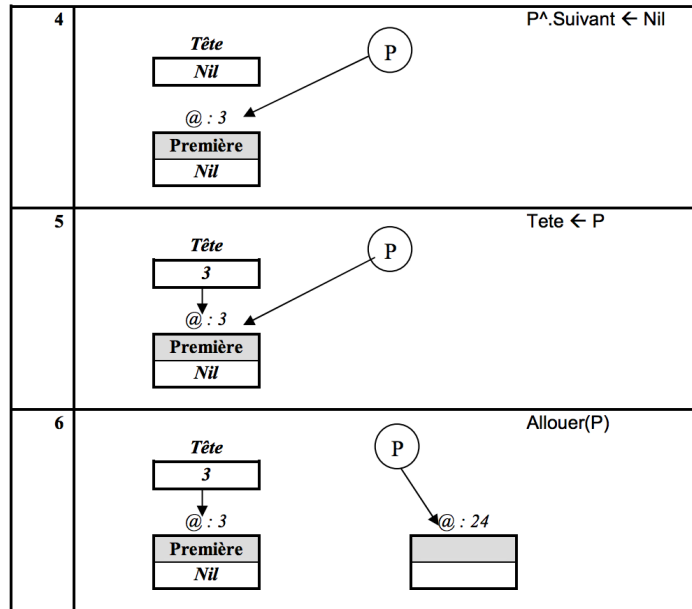
P^.Suivant ← Tete /* élément inséré en tête de liste */

Tete ← P

FIN



CRÉER UNE LISTE CHAÎNÉE



Algorithme CréationListe2Elements

Tete, P : Liste

NombreElt : entier

DEBUT

$Tete \leftarrow Nil$ /* pour l'instant la liste est vide */

$Allouer(P)$ /* réserve un espace mémoire pour le premier élément */

$Lire(P^{\wedge}.Info)$ /* stocke dans l'Info de l'élément pointé par P la valeur saisie */

$P^{\wedge}.Suivant \leftarrow Nil$ /* il n'y a pas d'élément suivant */

$Tete \leftarrow P$ /* le pointeur Tete pointe maintenant sur P */

/* Il faut maintenant ajouter le 2e élément, ce qui revient à insérer un élément en tête de liste */

$Allouer(P)$ /* réserve un espace mémoire pour le second élément */

$Lire(P^{\wedge}.Info)$ /* stocke dans l'Info de l'élément pointé par P la valeur saisie */

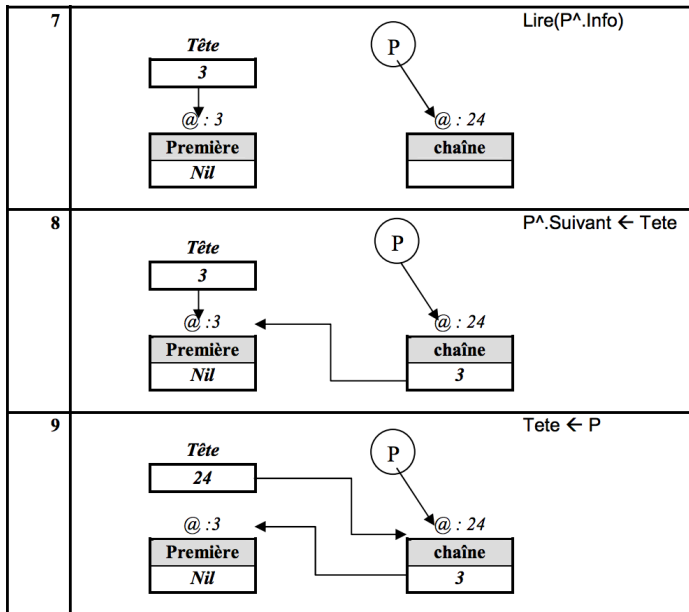
$P^{\wedge}.Suivant \leftarrow Tete$ /* élément inséré en tête de liste */

$Tete \leftarrow P$

FIN



CRÉER UNE LISTE CHAÎNÉE



Algorithme CréationListe2Elements

Tete, P : Liste

NombreElt : entier

DEBUT

Tete ← Nil /* pour l'instant la liste est vide */

Allouer(P) /* réserve un espace mémoire pour le premier élément */

Lire(P^.Info) /* stocke dans l'Info de l'élément pointé par P la valeur saisie */

P^.Suivant ← Nil /* il n'y a pas d'élément suivant */

Tete ← P /* le pointeur Tete pointe maintenant sur P */

/* Il faut maintenant ajouter le 2e élément, ce qui revient à insérer un élément en tête de liste */

Allouer(P) /* réserve un espace mémoire pour le second élément */

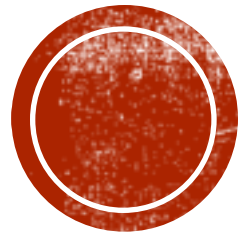
Lire(P^.Info) /* stocke dans l'Info de l'élément pointé par P la valeur saisie */

P^.Suivant ← Tete /* élément inséré en tête de liste */

Tete ← P

FIN



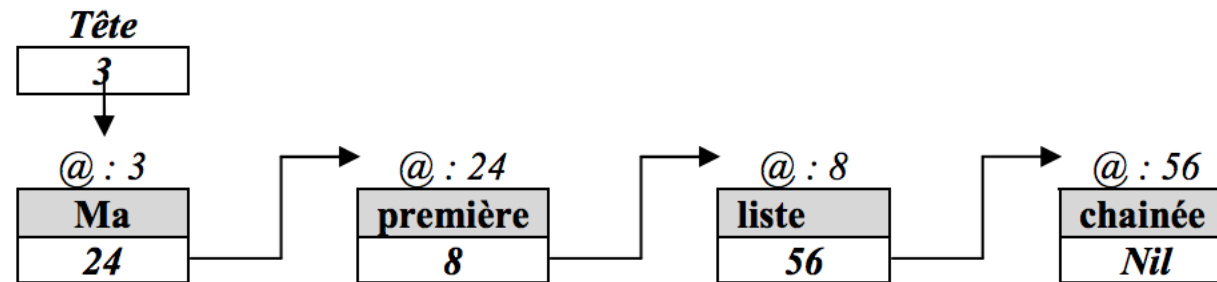


AFFICHER UNE LISTE CHAINÉE



AFFICHER LES ÉLÉMENTS D'UNE LISTE CHAÎNÉE

Une liste chaînée simple ne peut être parcourue que du premier vers le dernier élément de la liste.



AFFICHER LES ÉLÉMENTS D'UNE LISTE CHÂÎNÉE

DEBUT

$P \leftarrow \text{Tete}$ /* P pointe sur le premier élément de la liste*/

/* On parcourt la liste tant que l'adresse de l'élément suivant n'est pas Nil */

TANT QUE $P \neq \text{NIL}$ FAIRE /* si la liste est vide Tete est à Nil */

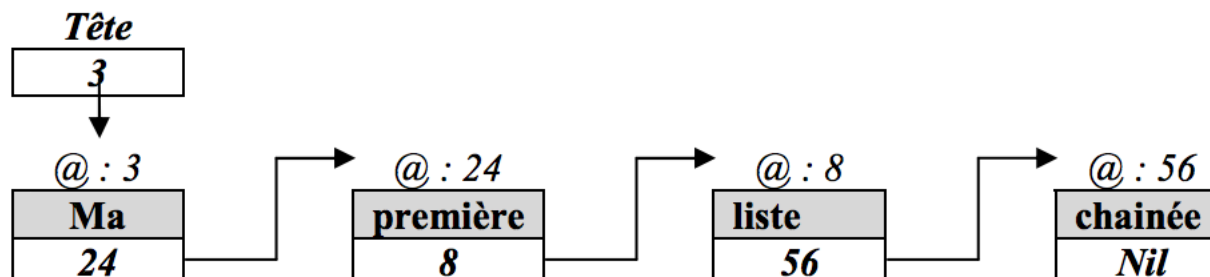
Ecrire(P^{Info}) /* afficher la valeur contenue à l'adresse pointée par P */

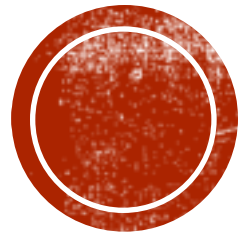
$P \leftarrow P^{\text{Suivant}}$ /* On passe à l'élément suivant */

FIN TANT QUE

FIN

1	P a pour valeur 3
2	"Ma" s'affiche
3	P prend pour valeur 24
2	"première" s'affiche
3	P prend pour valeur 8
2	"liste" s'affiche
3	P prend pour valeur 56
2	"chaînée" s'affiche
3	P prend pour valeur Nil





SUPPRIMER UN ÉLÉMENT DANS UNE LISTE CHAÎNÉE

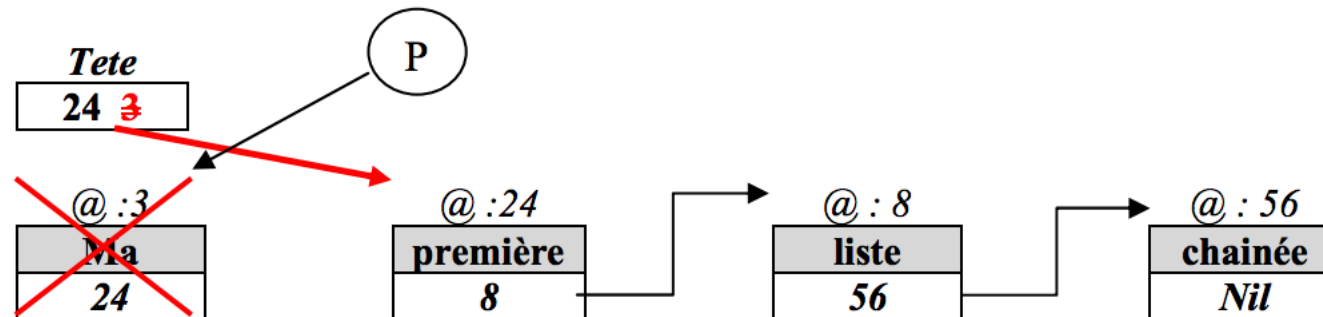


SUPPRIMER LE PREMIER ÉLÉMENT D'UNE LISTE CHAÎNÉE

Il y a deux actions, dans cet ordre, à réaliser :

- faire pointer la tête de liste sur le deuxième élément de la liste,
- libérer l'espace mémoire occupé par l'élément supprimé.

Il est nécessaire de déclarer un pointeur local qui va pointer sur l'élément à supprimer, et permettre de libérer l'espace qu'il occupait.



SUPPRIMER LE PREMIER ÉLÉMENT D'UNE LISTE CHAÎNÉE

DEBUT

SI Tete <> Nil **ALORS**

P ← Tete

Tete ← P^.Suivant

Desallouer(P)

/* la liste n'est pas vide on peut donc supprimer le premier élément */

/* P pointe sur le 1^{er} élément de la liste */

/* la tête de liste doit pointer sur le deuxième 'élément */

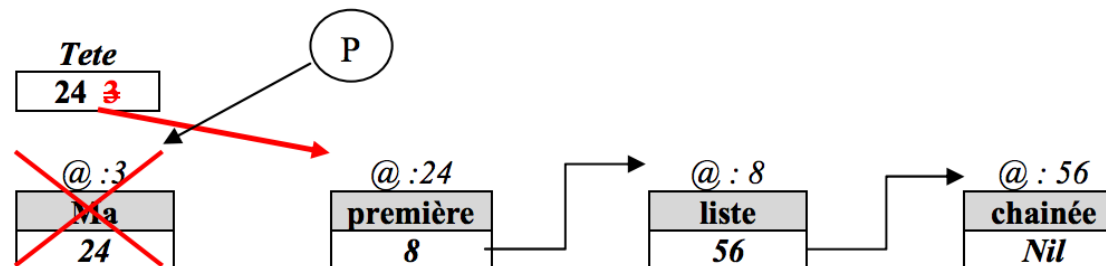
/* libération de l'espace mémoire qu'occupait le premier élément */

SINON

Ecrire("La liste est vide")

FINSI

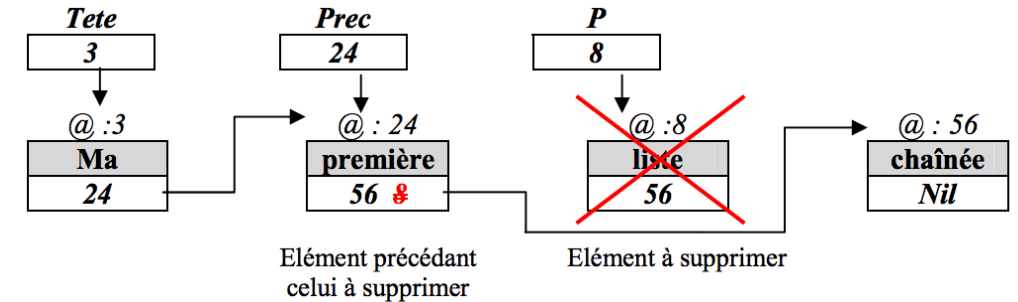
FIN



SUPPRIMER D'UNE LISTE CHAÎNÉE UN ÉLÉMENT PORTANT UNE VALEUR DONNÉE

Il faut:

- traiter à part la suppression du premier élément car il faut modifier le pointeur de tête,
- trouver l'adresse P de l'élément à supprimer,
- sauvegarder l'adresse Prec de l'élément précédant l'élément pointé par P pour connaître l'adresse de l'élément précédant l'élément à supprimer, puis faire pointer l'élément précédent sur l'élément suivant l'élément à supprimer,
- Libérer l'espace mémoire occupé par l'élément supprimé. L'exemple considère que l'on souhaite supprimer l'élément contenant la valeur "liste" de la liste ci-dessus.



SUPPRIMER D'UNE LISTE CHAÎNÉE UN ÉLÉMENT PORTANT UNE VALEUR DONNÉE

```

P      : Liste          /* pointeur sur l'élément à supprimer */
Prec   : Liste          /* pointeur sur l'élément précédant l'élément à supprimer */
Trouvé : Liste          /* indique si l'élément à supprimer a été trouvé */

DEBUT
  SI Tete <> Nil ALORS /* la liste n'est pas vide on peut donc y chercher une valeur à supprimer */
    SI Tete^.info = Val ALORS /* l'élément à supprimer est le premier */
      P ← Tete
      Tete ← Tete^.Suivant
      Desallouer(P)
    SINON /* l'élément à supprimer n'est pas le premier */
      Trouvé ← Faux
      Prec ← Tete /* pointeur précédent */
      P ← Tete^.Suivant /* pointeur courant */
      TANTQUE P <> Nil ET Non Trouvé
        SI P^.Info = Val ALORS /* L'élément recherché est l'élément courant */
          Trouvé ← Vrai
          SINON /* L'élément courant n'est pas l'élément cherché */
            Prec ← P /* on garde la position du précédent */
            P ← P^.Suivant /* on passe à l'élément suivant dans la liste */
          FINSI
        FIN TANT QUE
      SI Trouvé ALORS
        Prec^.Suivant ← P^.Suivant /* on "saute" l'élément à supprimer */
        Desallouer(P)
      SINON
        Ecrire ("La valeur ", Val, " n'est pas dans la liste")
      FINSI
    FINSI
  SINON
    Ecrire("La liste est vide")
  FINSI
FIN
  
```

